# Software Requirements Specification

SwishVision: Basketball Analytics Platform

Mallick Mikaal Imam  Minhaj ul Hassan  Muhammad Ahmed Silat

Syed Muhammad Sameer Hassan

Version 1.10
11th May 2026

| **Project Team** | |
|---|---|
| Mallick Mikaal Imam | |
| Minhaj ul Hassan | |
| Muhammad Ahmed Silat | |
| Syed Muhammad Sameer Hassan | |

| **Submission Date** | 11th May 2026 |
|---|---|

## Document History

| Version | Name | Date | Description of Change |
|---|---|---|---|
| 1.00 | Ahmed Silat | 16 Mar 2026 | Initial draft, project proposal translated to SRS |
| 1.10 | Minhaj ul Hassan | 11 May 2026 | Post-implementation update, locked in tech stack (FastAPI / React / SQL Server / Celery+Redis / JWT), reflected delivered scope, marked all use cases as implemented |
| 1.11 | Minhaj ul Hassan | 11 May 2026 | Corrected output codec to H.264 MP4 (transcoded via `imageio-ffmpeg`); added `imageio-ffmpeg`, `supervision`, and `pandas` to the Software Interfaces inventory |

## Distribution List

| Name | Role |
|---|---|
| | Supervisor / Course Instructor |
| | Project Manager |

## Document Sign-Off

| Version | Sign-off Authority | Project Role | Sign-off Date |
|---|---|---|---|
| 1.00 | | Supervisor | |

# Contents

# 1 Introduction

## 1.1 Purpose of Document

The purpose of this document is to outline the requirements for the construction of SwishVision, a web-based basketball analytics platform. It documents functional and non-functional requirements, system constraints, interface requirements, design constraints, and other factors vital to ensuring the software meets its intended goals. It serves as the primary reference for development, validation, and stakeholder alignment.

## 1.2 Intended Audience

The intended audience includes the course supervisor, the project development team, and any future maintainers of the system. The development team requires this document to ensure that implementation efforts are aligned with the stated purpose of the software. Supervisors and evaluators use it to assess the completeness and clarity of requirements. Future developers or contributors may reference it to understand system boundaries and expected behaviors.

## 1.3 Document Convention

- Font: Arial (Latin Modern as LaTeX substitute)
- Size: 11
- Line Spacing: 1.5

# 2 Overall System Description

## 2.1 Project Background

Basketball players at the amateur and semi-professional level lack access to the kind of analytical feedback that professional athletes receive from coaching staff and sports scientists. Critical aspects of shooting mechanics, such as release angle, elbow alignment, and follow-through consistency, occur too rapidly to be reliably analyzed by the naked eye. Furthermore, there exist no affordable, accessible tools that can automatically process practice footage and return structured, data-driven feedback.

SwishVision addresses this gap by building a web-based platform that accepts uploaded basketball footage and returns detailed performance analytics powered by computer vision and pose estimation. A fully working command-line prototype already exists (`main.py`), capable of detecting ball trajectory, rim interaction, and player skeletal keypoints using YOLOv8-based models. The object detection model (`best.pt`) was fine-tuned over 13 training iterations on the Basketball-1 dataset sourced from Roboflow (Eagle Eye workspace, CC BY 4.0), which provides labeled bounding boxes for three classes: basketball, rim, and sports ball. Human pose

estimation uses the pre-trained `yolov8m-pose.pt` model, tracking 17 body joint keypoints per player.

### 2.2 Project Scope

- Allow users to create accounts and upload basketball practice footage via a web interface.
- Process uploaded video using the existing CV pipeline: ball tracking, rim tracking, and human pose estimation.
- Detect shot attempts and classify them as made or missed based on ball-rim interaction.
- Analyze shooting form using skeletal keypoints and compute joint angles across frames.
- Generate a per-session report including shot percentage, consistency scores, and mechanical feedback.
- Display session history and allow longitudinal comparison of performance across sessions.
- Provide a personal analytics dashboard summarizing trends over time.

### 2.3 Not In Scope

- Real-time (live) video analysis, the system processes pre-recorded footage only.
- Multi-player team analytics, the current scope focuses on individual player analysis.
- Mobile native applications (iOS/Android), the platform is web-only.
- Analysis of non-shooting basketball actions such as dribbling, defense, or passing.

### 2.4 Project Objectives

The primary objective of SwishVision is to make professional-grade shooting analytics widely accessible for amateur basketball players. Specific objectives include:

- Automate the detection and classification of shot attempts with high accuracy.
- Provide quantifiable metrics for shooting form that are consistent and reproducible.
- Enable players to track their progress over time through a persistent session history.
- Present results in a format that is intuitive and actionable for non-technical users.
- Expose the underlying CLI pipeline through a web interface without requiring users to have any technical knowledge.

## 2.5 Stakeholders

**Amateur Basketball Players (End Users).** The primary users of the system. They upload footage, view their analytics, and act on the feedback provided. Their goal is to improve shooting form and track progress.

**Development Team.** Responsible for building the system. Must ensure the web platform correctly integrates with the existing CV pipeline and delivers results reliably.

**Course Supervisor.** Evaluates the project from an academic standpoint. Requires documentation, working demonstrations, and compliance with software engineering standards.

## 2.6 Operating Environment

- **Server:** Linux or Windows environment running Python 3.8+, supporting PyTorch and YOLOv8 inference.
- **Client:** Any modern web browser (Chrome, Firefox, Safari, Edge), no installation required.
- **GPU:** CUDA-compatible NVIDIA GPU strongly recommended on the server side (PyTorch 2.0+); CPU fallback supported but slower.
- **RAM:** Minimum 4 GB; 8 GB+ recommended.
- **Storage:** Minimum 10 GB for model files, uploaded videos, and generated outputs.
- **Network:** Standard HTTPS internet connection for the client.
- **Output Codec:** H.264 in MP4 container, frames are written with OpenCV's `mp4v` fourcc and then transcoded to H.264 (`yuv420p`, `+faststart`) via `imageio-ffmpeg` so the result plays directly in `<video>` elements in all major browsers. If `imageio-ffmpeg` is unavailable the pipeline keeps the `mp4v` output as a fallback.

## 2.7 System Constraints

- Video processing is computationally intensive; processing time scales with video length and resolution.
- Accuracy of shot detection and pose estimation is constrained by video quality, camera angle, and lighting.
- The system relies on pre-trained YOLOv8 models (`best.pt` for ball/rim, `yolov8m-pose.pt` for human pose).
- The web application must handle asynchronous video processing, the upload and the results must be decoupled with a job queue or background worker.

### 2.8 Assumptions & Dependencies

- Uploaded videos are recorded from a side-on or slightly elevated angle capturing both player and basket.
- The player being analyzed occupies the majority of the frame.
- The server environment has Python 3.8+, PyTorch 2.0+, Ultralytics YOLOv8, OpenCV, NumPy, and Matplotlib installed.
- Model files (`models/best.pt` and `models/yolov8m-pose.pt`) are present on the server.
- Users have a device capable of recording and uploading video.
- The existing CLI pipeline is treated as a stable subsystem, wrapped, not rewritten, by the web layer.
- Intermediate data files (`angs.txt`, `xy_coords.txt`, `detections.txt`, `ball_locl.txt`) are transient.
- Text reports are generated per video and stored in `reports/`.

## 3 Software Development Methodology

An **Incremental Development** methodology will be used. This choice is appropriate because:

- A functioning core (the CV pipeline) already exists and can serve as the first increment.
- Requirements for the web layer can be developed and validated progressively.
- Early increments can be demonstrated to the supervisor for feedback before more advanced features are built.
- The team can parallelize work on the backend processing pipeline and the frontend interface.

**Delivered Increments (all completed as of 11 May 2026):**

| Increment | Deliverable | Status |
|---|---|---|
| 1 | User registration/login + video upload endpoint | Delivered |
| 2 | Background processing integration, pipeline runs on uploaded video via Celery | Delivered |
| 3 | Shot detection results displayed on frontend (React) | Delivered |
| 4 | Pose/form analysis results and report generation | Delivered |
| 5 | Session history and longitudinal dashboard | Delivered |

# 4 System Analysis & Requirement Gathering

Requirements were gathered through the following approaches:

- **Prototype Analysis:** The existing CLI pipeline (`main.py`) was examined to understand its outputs, ball tracks, rim tracks, human keypoints, joint angles, ball-hand contact frames, shot start events, and output video.
- **README Review:** The project README was reviewed to extract the module inventory, model files, dataset information, and the pipeline's 12-step processing flow.
- **Output Inspection:** Sample reports and annotated videos in `output_videos/` were reviewed.
- **Stub File Analysis:** `stubs_utils.py` and intermediate files indicate the pipeline supports caching, which is reused in the web architecture.
- **Project Proposal Review:** The proposal was analyzed to extract goals, features, and value propositions.
- **Gap Analysis:** Differences between CLI outputs and web user needs drove web-specific requirements (auth, upload, async processing, results, dashboard).

# 5 External Interface Requirements

## 5.1 Hardware Interfaces

- Server machine with sufficient CPU/RAM for Python inference. CUDA-compatible NVIDIA GPU strongly recommended.
- End users require only a standard computer or smartphone with internet access and a web browser.
- A camera or smartphone capable of recording at minimum 720p resolution is recommended.

## 5.2 Software Interfaces

- **Python 3.8+**: core runtime for the CV pipeline.
- **Ultralytics YOLOv8** (`ultralytics>=8.0.0`).
- **PyTorch 2.0+** (`torch>=2.0.0`, `torchvision>=0.15.0`).
- **OpenCV** (`opencv-python>=4.8.0`), video reading, frame processing, initial frame writing (`mp4v` fourcc).
- **imageio-ffmpeg**: bundles a static `ffmpeg` binary used to transcode to browser-playable H.264 MP4.

- **supervision** + **pandas**: used by the trackers.
- **NumPy** (`numpy>=1.24.0`).
- **Matplotlib** (`matplotlib>=3.7.0`).
- **FastAPI** (`fastapi==0.135.3`, `uvicorn==0.42.0`).
- **React** (Create React App, Node.js 18+).
- **SQLAlchemy** 2.0 + **Alembic** 1.14.
- **Microsoft SQL Server 2019+** with **pyodbc** 5.3 (ODBC Driver 17).
- **Celery** 5.6 + **Redis** 7+.
- **JWT** (`python-jose`, `passlib[bcrypt]`, `bcrypt`).
- **Web Browser**: Chrome, Firefox, Safari, or Edge.

### 5.3 Communications Interfaces

- **HTTPS**: all client-server communication encrypted via TLS.
- **REST API**: frontend communicates with backend via HTTP/REST for upload, status polling, and results.
- **WebSocket or Polling**: for job status updates.

## 6 Functional Requirements

### 6.1 Data Flow Diagrams

#### Context Level DFD

The user submits a video and receives back analytics. The platform internally invokes the CV Engine and interacts with the database.

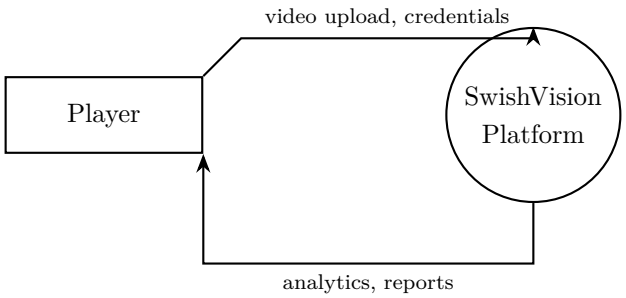


*Figure 1: Context Level DFD*

**Level 0 DFD**

The system decomposes into five major processes: Authentication, Video Upload, CV Processing, Report Generation, and Dashboard.

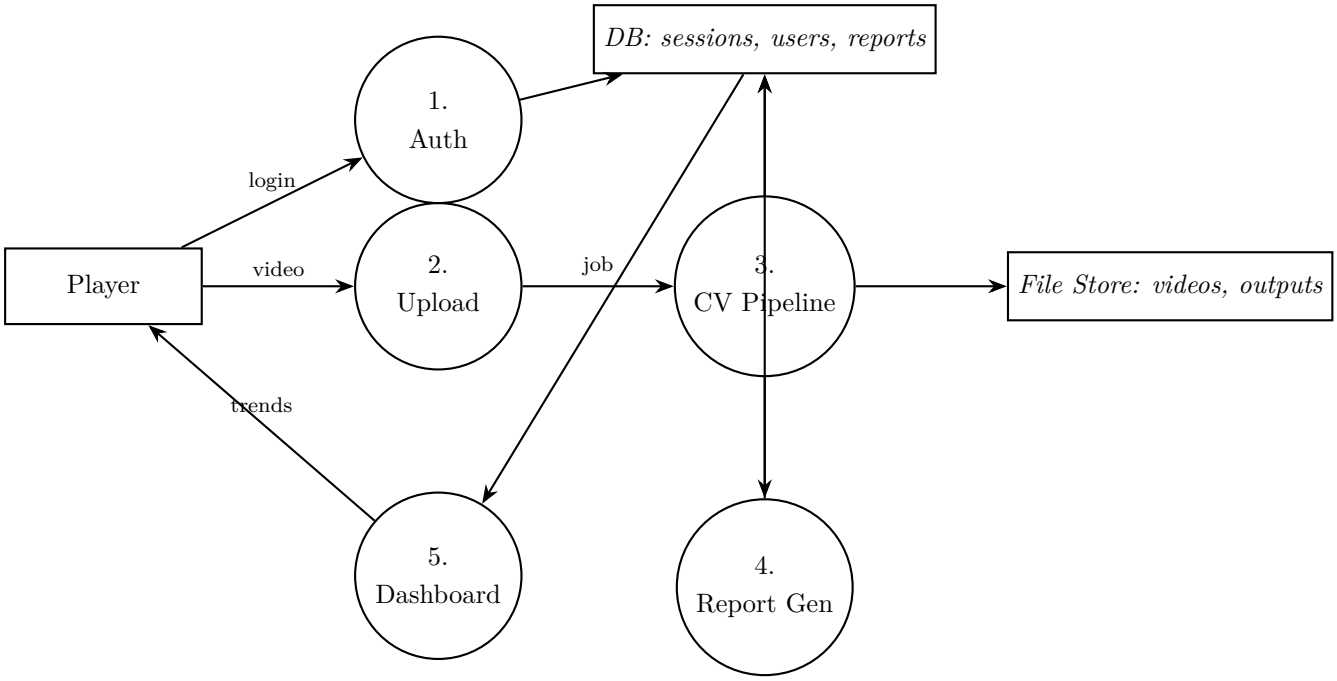


*Figure 2: Level 0 DFD*

**Level 1 DFD: Process 3: CV Processing Pipeline**

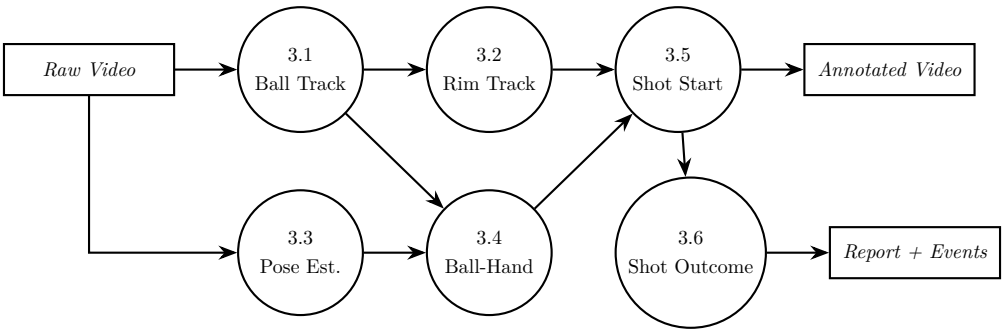


*Figure 3: Level 1 DFD: CV Pipeline*

## 6.2 CRUD and Association Matrices

**Data to Location CRUD Matrix:**

| **Entity.Attribute** | **Player (Self)** | **System (Backend)** |
|---|---|---|
| **User** | | |

| | | |
|---|---|---|
| .UserID | R | CRUD |
| .Email | RU | CRUD |
| .PasswordHash | | CRUD |
| **Session** | | |
| .SessionID | R | CRUD |
| .UserID | R | R |
| .UploadDate | R | CRUD |
| .VideoPath | | CRUD |
| .Status | R | CRUD |
| **Report** | | |
| .ReportID | R | CRUD |
| .ShotPercentage | R | CRU |
| .ConsistencyScore | R | CRU |
| .FormFeedback | R | CRU |
| **ShotEvent** | | |
| .FrameIndex | R | CRU |
| .Outcome | R | CRU |
| .ReleaseAngle | R | CRU |

**Process to Location Association Matrix:**

| **Process** | **Player** | **Backend** |
|---|---|---|
| Register / Login | X | X |
| Upload Video | X | X |
| Trigger Processing | | X |
| View Results | X | X |
| View Session History | X | X |
| Delete Session | X | |

## 6.3 Use Cases

### Use Case Diagram

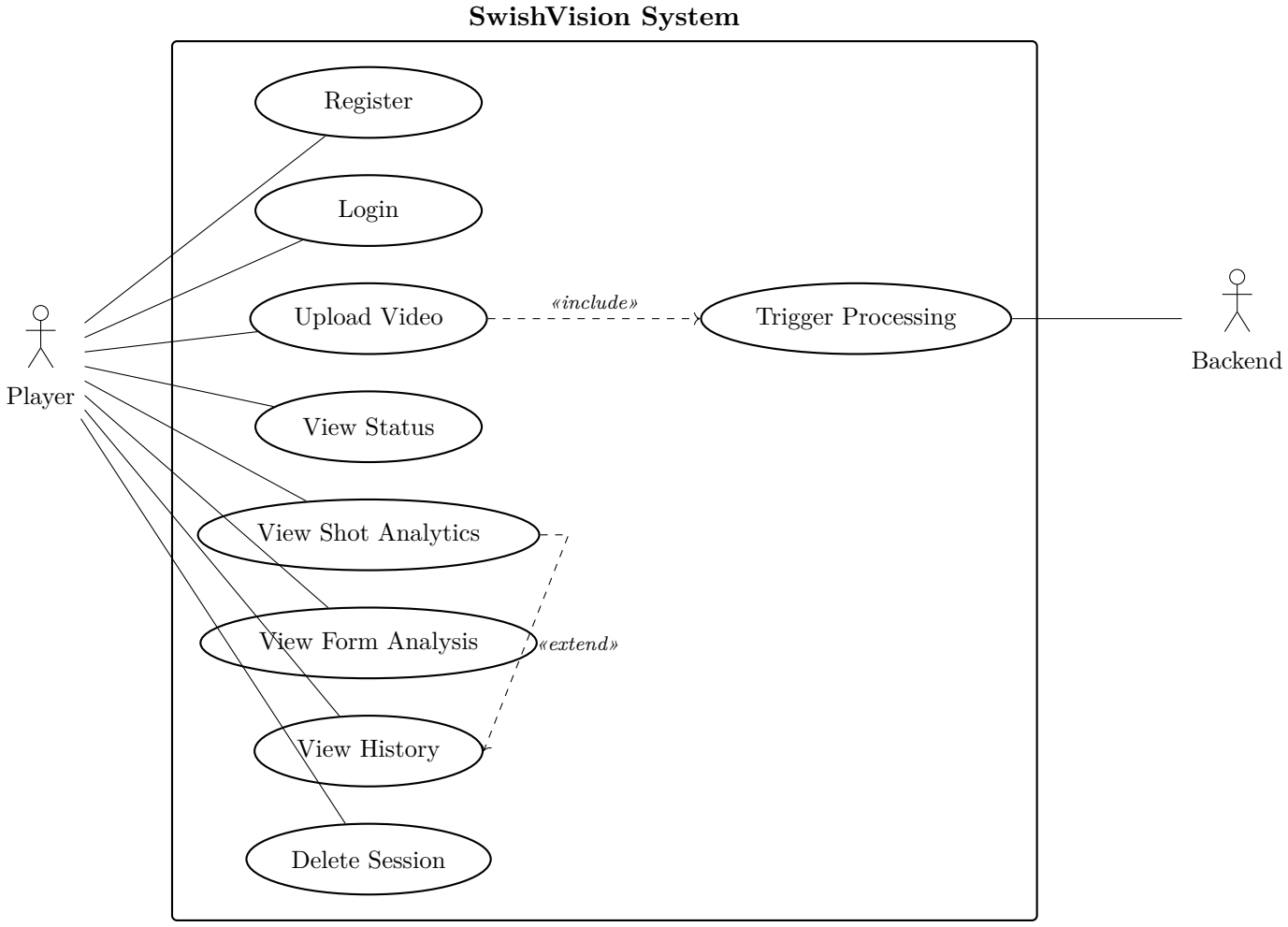


*Figure 4: Use Case Diagram*

**Use Case 1: User Registration**

| | |
|---|---|
| Use-Case Name | User Registration |
| Use-Case ID | SV-01 |
| Type | Business Requirement |
| Priority | High |
| Primary Actor | Player |
| Other Actors | System Backend |
| Description | A new user creates an account by providing name, email, and password. The system validates inputs and creates the account. |
| Precondition | The user does not already have an account with the provided email. |
| Trigger | User navigates to the registration page and submits the form. |

| | |
|---|---|
| Typical Course | **Actor:** provides name, email, password. **System:** validates, hashes password, stores user record, redirects to dashboard. |
| Alternate Courses | Alt 1: Email exists: error + prompt login. Alt 2: Weak password: rejected with requirements. |
| Conclusion | User account is created successfully. |
| Post Condition | User is logged in and redirected to their dashboard. |

**Use Case 2: Upload Practice Video**

| | |
|---|---|
| Use-Case Name | Upload Practice Video |
| Use-Case ID | SV-02 |
| Type | Business Requirement |
| Priority | High |
| Primary Actor | Player |
| Other Actors | System Backend |
| Description | A logged-in user uploads a basketball practice video file. The system accepts the file, stores it, and enqueues it for processing. |
| Precondition | User is authenticated. |
| Trigger | User selects a video file and clicks Upload. |
| Typical Course | **Actor:** selects video, submits. **System:** validates format/size, stores video, creates Session (status=queued), returns job ID. |
| Alternate Courses | Alt 1: Unsupported format: rejected. Alt 2: File too large: rejected with limit message. |
| Conclusion | Video stored and queued for processing. |
| Post Condition | Session record created with status “queued”. |

**Use Case 3: Process Video (CV Pipeline)**

| | |
|---|---|
| Use-Case Name | Process Video |
| Use-Case ID | SV-03 |
| Type | System |
| Priority | High |
| Primary Actor | System Backend |

| | |
|---|---|
| Description | The backend worker picks up a queued video job and runs the CV pipeline: ball/rim tracking, pose estimation, shot detection, and report generation. |
| Precondition | A video has been uploaded and a job is queued. |
| Trigger | Background worker dequeues the job. |
| Typical Course | Runs `main_pipeline()` on the video. Generates annotated output video, shot events, joint angle data, report text. Updates Session status to "completed". |
| Alternate Courses | Alt 1: Inference error: status=failed, error logged. Alt 2: No shots detected: report generated with zero shots. |
| Conclusion | Session results stored, status updated. |
| Post Condition | Session status "completed"; Report and ShotEvent rows created. |

**Use Case 4: View Shot Analytics**

| | |
|---|---|
| Use-Case Name | View Shot Analytics |
| Use-Case ID | SV-04 |
| Type | Business Requirement |
| Priority | High |
| Primary Actor | Player |
| Description | The user views results of a processed session: shot percentage, annotated video, per-shot outcomes, consistency score. |
| Precondition | Session status = completed. |
| Trigger | User clicks on a completed session. |
| Typical Course | **Actor:** selects session. **System:** displays shot percentage, made/missed breakdown, video player, consistency score. |
| Alternate Courses | Alt 1: Still processing: progress indicator. Alt 2: Failed: failure message with guidance. |
| Conclusion | User reviews shot data. |
| Post Condition | No state change. |

**Use Case 5: View Form Analysis**

| | |
|---|---|
| Use-Case Name | View Form Analysis |
| Use-Case ID | SV-05 |

| | |
|---|---|
| Type | Business Requirement |
| Priority | High |
| Primary Actor | Player |
| Description | User views biomechanical analysis: release angle, elbow alignment feedback, consistency rating derived from joint angles. |
| Precondition | Session processed; pose data available. |
| Trigger | User navigates to Form tab of a completed session. |
| Typical Course | Displays per-shot joint angle charts, avg release angle, consistency, plain-language feedback. |
| Alternate Courses | Alt 1: Pose data unavailable: message indicating insufficient data. |
| Conclusion | User gains insight into shooting form. |
| Post Condition | None. |

**Use Case 6: View Session History**

| | |
|---|---|
| Use-Case Name | View Session History |
| Use-Case ID | SV-06 |
| Type | Business Requirement |
| Priority | Medium |
| Primary Actor | Player |
| Description | User views a chronological list of all past sessions with summary stats and can compare metrics over time. |
| Precondition | User has at least one completed session. |
| Trigger | User navigates to History/Dashboard. |
| Typical Course | Displays sessions sorted by date with shot percentage, consistency score, trend chart. |
| Alternate Courses | Alt 1: No sessions: prompt to upload first video. |
| Conclusion | User views longitudinal progress. |
| Post Condition | None. |

**Use Case 7: Delete Session**

| | |
|---|---|
| Use-Case Name | Delete Session |
| Use-Case ID | SV-07 |

| Type | Business Requirement |
|---|---|
| Priority | Low |
| Primary Actor | Player |
| Description | User permanently deletes a session and all associated data. |
| Precondition | Session belongs to authenticated user. |
| Trigger | User clicks Delete on session entry. |
| Typical Course | **Actor:** confirms deletion. **System:** removes video, report, DB records. |
| Alternate Courses | Alt 1: Session processing: deletion rejected until done. |
| Conclusion | Session and related data removed. |
| Post Condition | Session no longer in history. |

# 7 Non-Functional Requirements

## 7.1 Performance Requirements

- Video processing should complete within a reasonable duration relative to video length, a 1-minute video within 5 minutes on a GPU server.
- Frontend interactions (page loads, navigation) should respond within 2 seconds under normal load.
- File upload should support videos up to 500 MB without timeout.

## 7.2 Safety Requirements

- Uploaded video files stored in an isolated server directory not directly accessible via URL.
- Failed processing jobs caught and logged without crashing the worker.
- Output files verified for existence before being served to the user.
- Original uploaded video preserved on processing failure.

## 7.3 Security Requirements

- User passwords stored as bcrypt hashes.
- All user-data endpoints verify authentication and resource ownership.
- Upload endpoints validate MIME type and file extension.

- HTTPS enforced in production.
- Endpoints protected against common vulnerabilities.

### 7.4 User Documentation

- In-app help/onboarding for recording suitable videos and interpreting results.
- Developer README for environment setup and running the app.
- Inline tooltips for key metrics (Consistency Score, Release Angle).

## 8 References


- YOLOv8 Documentation (Ultralytics): https://docs.ultralytics.com
- Project Proposal: SwishVision
- Existing CLI codebase: `main.py` and associated tracker/drawer modules
- IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications


## 9 Appendices

### Appendix A: Glossary

| Term | Definition |
|---|---|
| YOLOv8 | Real-time object detection model from Ultralytics, used here for ball, rim, and pose detection. |
| Pose Estimation | CV technique that detects and tracks positions of human body joints (keypoints) in video frames. |
| Release Angle | The angle of the ball's trajectory at the point it leaves the player's hands. |
| Consistency Score | Derived metric indicating how uniform a player's shooting form is across attempts in a session. |
| Shot Event | A detected shot attempt, frame, trajectory, outcome (made/missed). |
| CV Pipeline | Chain of CV processing steps: ball tracking → rim tracking → pose estimation → shot detection → report. |
| Session | A single uploaded video and all data derived from its processing. |